UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL RESISTENCIA (UTN FRRe)
Tecnicatura Universitaria en Logística
Extensión Ituzaingó (Corrientes)

Informática Aplicada
Prof. Ing. Juan Manuel Goya

Segunda etapa: la logística de la empresa y su aplicación web

Etapa 2 — Empresa de bicicletas compartidas

Alumno: Santiago T. Sala
Legajo: 30794
Fecha: 12 de octubre de 2026

Trabajo sobre la misma empresa de la Etapa 1. Agregué funciones logísticas y actualicé el modelo de datos. Están los seis puntos de la consigna: el SQL lo corrí en Supabase y la aplicación web está publicada en Vercel (https://bicicletas-compartidas.vercel.app). Todos los datos son ficticios.

1. Funciones logísticas que elegí

#	Función	Problema que resuelve	Quién la usa	Datos que necesita	Resultado o decisión
F1	Disponibilidad y alertas por estación	Usuarios que llegan a una estación vacía o llena (los cuellos de botella de la Etapa 1)	Usuario y supervisor	Bicis por estado y estación, capacidad máxima, umbral mínimo y máximo	Lista de estaciones marcadas como VACIA, OK o LLENA; activa la F2
F2	Redistribución de bicicletas	Las bicis se acumulan en algunas estaciones y faltan en otras	Supervisor (crea la orden) y operario (la hace)	Alertas de F1, estación de salida y de llegada, bicis a mover, operario	Orden de traslado (PENDIENTE, EN_CURSO, COMPLETADA) e historial de qué bici se movió
F3	Mantenimiento preventivo y reportes de falla	Bicis rotas que se siguen prestando y tareas sin planificar	Usuario (reporta), supervisor (programa), operario (repara)	Reportes de falla, mantenimientos (tipo, estado, bici), operarios	Lista de tareas pendientes por operario y por fecha; la bici pasa a EN_MANTENIMIENTO
F4	Control de repuestos	Reparaciones frenadas porque faltan repuestos	Operario (usa) y supervisor (compra)	Repuestos, stock actual y mínimo, consumo por mantenimiento	Descuento automático del stock y lista de repuestos a reponer
F5	Indicadores	Hoy no se mide la operación	Supervisor	Viajes, bicis, alertas, reportes	Viajes terminados, duración promedio, % de bicis en mantenimiento, estaciones con alerta y fallas abiertas

2. Documento descriptivo del negocio (texto que uso como prompt)

Contexto de la empresa. BiciTUL (nombre ficticio) es una empresa de bicicletas compartidas en una ciudad mediana del NEA. Tiene estaciones fijas con anclajes en lugares de mucho movimiento (plaza central, costanera, terminal). Los usuarios registrados retiran una bicicleta en una estación y la devuelven en cualquier otra. Los problemas logísticos son:

- estaciones que se vacían (sobre todo a la mañana, en los barrios) o se llenan (en el centro);
- bicicletas con fallas que se siguen prestando;
- reparaciones que se demoran por falta de repuestos;

- ninguna medición de cómo funciona la operación.

Actores y roles.

1. *Usuario:* se registra, carga sus tarjetas, ve la disponibilidad, inicia y termina viajes y reporta fallas.
2. *Operario técnico:* ve sus tareas de mantenimiento y sus órdenes de traslado, las marca como hechas y registra los repuestos que usó.
3. *Supervisor de operaciones:* administra estaciones, bicicletas, operarios y repuestos, crea órdenes de traslado, programa mantenimientos y ve los indicadores.

Datos del negocio (según el MER de la Etapa 2).

- ESTACION (codigo, direccion, capacidad_max, latitud, longitud, umbral_minimo, umbral_maximo).
- BICICLETA (numero_serie, estado: DISPONIBLE / EN_USO / EN_MANTENIMIENTO / EN_TRASLADO, fecha_alta, estación actual).
- USUARIO (dni, nombre).
- TARJETA (id, marca, últimos 4 dígitos, vencimiento; pertenece a un usuario).
- VIAJE (usuario, bicicleta, fecha_hora_inicio, fecha_hora_fin, estación de origen, estación de destino, tarjeta).
- REPORTE_FALLA (id, viaje, fecha_hora, descripción, estado ABIERTO / RESUELTO).
- MANTENIMIENTO (id, fecha, descripción, estación, operario, tipo PREVENTIVO / CORRECTIVO, estado PROGRAMADO / REALIZADO, bicicleta opcional).
- REPUESTO (código, descripción, stock_actual, stock_minimo).
- CONSUMO_REPUESTO (mantenimiento, repuesto, cantidad).
- REDISTRIBUCION (id, estación de salida, estación de llegada, operario, hora de salida, hora de llegada, estado).
- REDISTRIBUCION_BICICLETA (redistribución, bicicleta).

Funciones logísticas.

- **F1 Disponibilidad y alertas:** para cada estación se cuentan las bicis disponibles y las asignadas. La estación está VACIA si las disponibles son menos que el umbral mínimo, y LLENA si las asignadas superan el umbral máximo.
- **F2 Redistribución:**
  - El supervisor crea una orden desde una estación LLENA hacia una VACIA, y elige las bicis y el operario.
  - Al salir, las bicis pasan a EN_TRASLADO.
  - Al llegar, cambian de estación y vuelven a DISPONIBLE.
- **F3 Mantenimiento:**
  - Un reporte de falla genera un mantenimiento CORRECTIVO para esa bici, y la bici pasa a EN_MANTENIMIENTO.
  - Los mantenimientos PREVENTIVOS los programa el supervisor.
  - Cuando un mantenimiento se marca REALIZADO, la bici vuelve a DISPONIBLE y el reporte pasa a RESUELTO.
- **F4 Repuestos:** al registrar un consumo se descuenta el stock. Si el stock queda igual o por debajo del mínimo, el repuesto aparece en la lista “a reponer”.

- **F5 Indicadores:** tablero con los indicadores de la tabla del punto 1.

Reglas del negocio.

- Una estación nunca supera su capacidad máxima.
- Una bici solo se puede retirar si está DISPONIBLE.
- Una bici, o un usuario, no pueden tener dos viajes abiertos al mismo tiempo.
- El viaje termina después de empezar y se paga con una tarjeta del mismo usuario.
- Nunca se guarda el número completo de la tarjeta.
- El stock de repuestos no puede ser negativo.
- Un traslado no puede ir de una estación a la misma estación.
- Solo el supervisor da de alta estaciones, bicis, operarios y repuestos.
- El operario solo modifica sus propias tareas.
- El usuario solo ve y modifica sus datos y sus viajes.

Pantallas y reportes.

- *Usuario:* lista o mapa de estaciones con disponibilidad, retirar bici, devolver bici, mis viajes y reportar falla.
- *Operario:* mis tareas (mantenimientos y traslados) y registro de repuestos usados.
- *Supervisor:* altas, bajas y cambios de estaciones, bicis, operarios y repuestos; alertas por estación; crear orden de traslado; programar mantenimiento; repuestos a reponer; indicadores.

Datos de ejemplo.

- 3 estaciones: E01 Plaza Central (capacidad 6), E02 Costanera (capacidad 4) y E03 Terminal (capacidad 8).
- 9 bicis, 7 de ellas en E03 (por eso E03 queda LLENA y E01 VACIA).
- 2 usuarios ficticios con 3 tarjetas.
- 3 viajes (uno en curso).
- 1 reporte de falla, 2 mantenimientos, 2 repuestos y 1 orden de traslado.

Están en sql/etapa2_datos_y_pruebas.sql.

Requisitos técnicos.

- Aplicación web que se pueda usar desde el celular.
- Conectada a una base PostgreSQL en Supabase, creada con sql/etapa2_crear_base.sql.
- Pensada para publicarse en Vercel (por ejemplo, Next.js con supabase-js).
- Las credenciales van en variables de entorno (NEXT_PUBLIC_SUPABASE_URL y NEXT_PUBLIC_SUPABASE_ANON_KEY), nunca en el código.
- Permisos por rol con Row Level Security de Supabase.
- Pantallas en español.

3. Nuevo MER

En el diagrama marqué lo nuevo con borde grueso y la palabra [NUEVA]. Los atributos nuevos o modificados llevan un signo “+”.

Cambio	Tipo	Por qué lo hice
ESTACION: + latitud, longitud, umbral_minimo, umbral_maximo	modificada	Para la F1: saber cuándo una estación está vacía o llena, y ubicarla en un mapa
BICICLETA: + estado EN_TRASLADO, + fecha_alta	modificada	Para la F2: una bici que está en el camión no está disponible. Para la F3: la antigüedad sirve para planificar el preventivo
REDISTRIBUCION, con las relaciones desde, hacia, ejecuta (operario) y traslada (N:M con BICICLETA)	nueva	Para la F2: guardar el historial de traslados, que en la Etapa 1 se perdía
REPORTE_FALLA, con la relación se reporta en (N:1 con VIAJE)	nueva	Para la F3: la falla la informa el usuario al devolver la bici, y el viaje ya dice qué usuario y qué bici
MANTENIMIENTO: + tipo, + estado, relación se aplica a (con BICICLETA, opcional)	modificada	Para la F3: diferenciar preventivo de correctivo y pendiente de realizado, y saber qué bici se reparó
REPUESTO, con la relación consume (N:M con MANTENIMIENTO y atributo cantidad)	nueva	Para la F4: stock y consumo de repuestos

MER actualizado — Bicicletas compartidas (Etapa 2). Borde grueso y [NUEVA] = nuevo; + = atributo nuevo o modificado

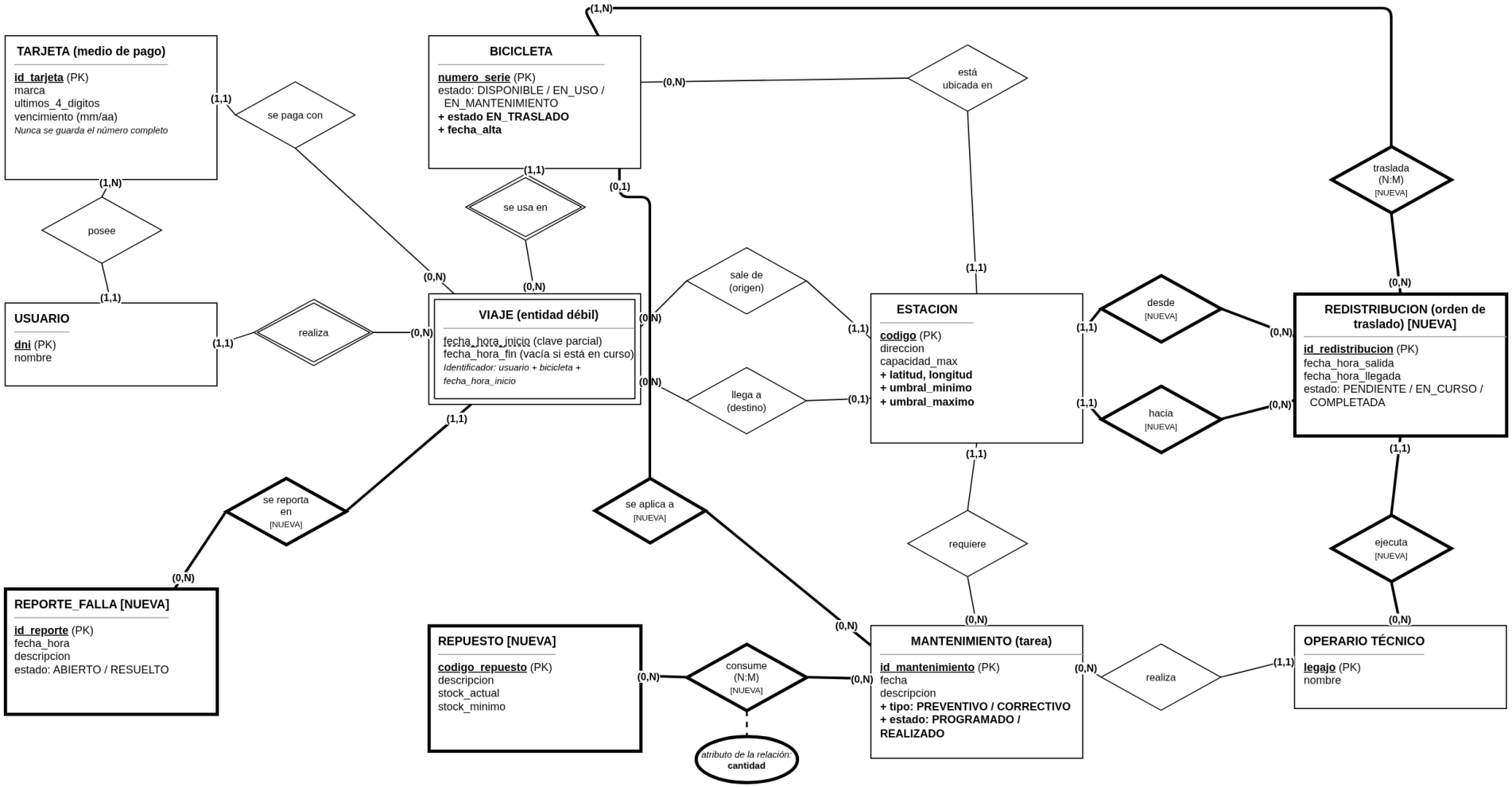


Figura 1. MER de la Etapa 2 (archivo editable: bicicletas_MER_etapa2.drawio)

4. MR y SQL con IA

Pedido a la IA:

> Con este nuevo MER [lista de entidades, atributos y relaciones del documento descriptivo], actualizá el modelo relacional y el script PostgreSQL para Supabase. Aplicá: N:M → tabla puente; atributos de la relación → columnas de la tabla puente; CHECK para todos los estados; trigger que descuente stock al consumir repuestos; vistas para disponibilidad por estación, mantenimientos pendientes, repuestos a reponer e indicadores. Decí si conviene recrear la base o modificarla.

Respuesta. Usé el mismo asistente de IA que en la Etapa 1 (Grok Bot). El script que me devolvió está completo en el Anexo A. [Si uso otra IA de la lista, reemplazo esta respuesta por la suya completa.]

Modelo relacional nuevo (marco con * lo nuevo):

- ESTACION (codigo PK, direccion, capacidad_max, *latitud,* longitud, *umbral_minimo,* umbral_maximo)
- BICICLETA (numero_serie PK, estado, codigo_estacion FK, *fecha_alta)
- USUARIO, TARJETA, OPERARIO y VIAJE: igual que en la Etapa 1
- *REPORTE_FALLA (id_reporte PK, (dni_usuario, numero_serie, fecha_hora_inicio) FK→VIAJE, fecha_hora, descripcion, estado)
- MANTENIMIENTO (…, *tipo,* estado, *numero_serie FK→BICICLETA opcional)
- *REPUESTO (codigo_repuesto PK, descripcion, stock_actual, stock_minimo)
- *CONSUMO_REPUESTO (id_mantenimiento FK, codigo_repuesto FK, cantidad; PK compuesta)
- *REDISTRIBUCION (id_redistribucion PK, codigo_estacion_desde FK, codigo_estacion_hacia FK, legajo_operario FK, fecha_hora_salida, fecha_hora_llegada, estado)
- *REDISTRIBUCION_BICICLETA (id_redistribucion FK, numero_serie FK; PK compuesta)

¿Recrear o modificar la base? Decidí recrearla desde cero, por tres razones:

- la base de la Etapa 1 solo tiene datos de prueba;
- así el script queda completo en un solo archivo y el profesor lo puede reproducir;
- evito los errores de aplicar cambios por partes.

Si hubiera datos reales, habría que modificar las tablas con ALTER TABLE para no perderlos.

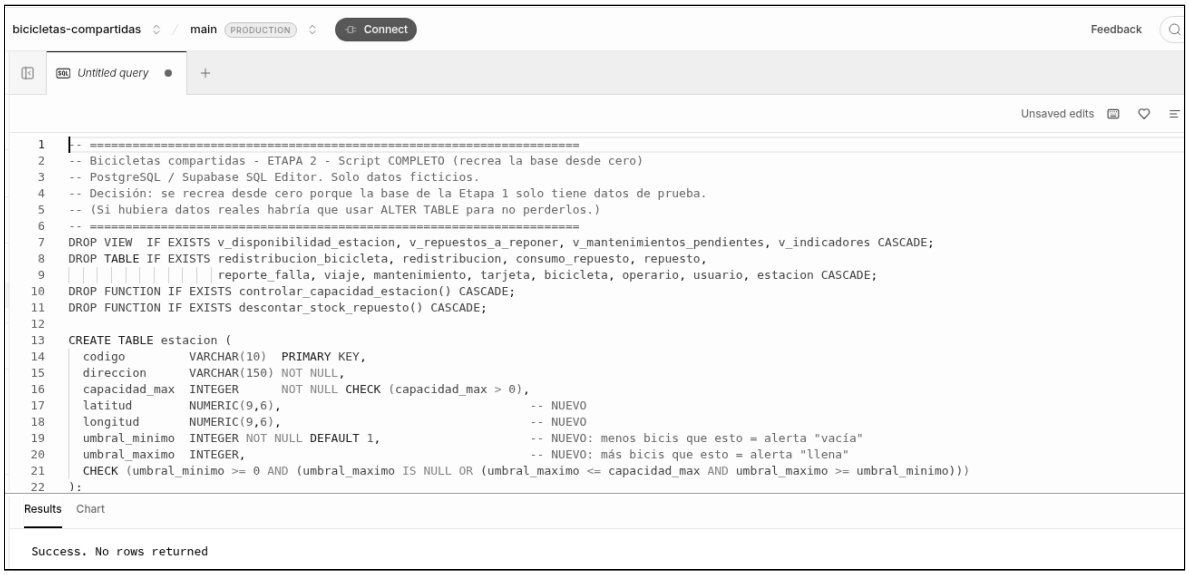
Revisión crítica. Coincide con mi nuevo MER: son 12 tablas, con 2 tablas puente para las relaciones N:M. Observé lo siguiente:

1. La clave foránea de REPORTE_FALLA hacia VIAJE tiene 3 columnas, porque VIAJE es entidad débil. Funciona, pero para la aplicación quizás convenga un id_viaje interno.
2. El cambio automático del estado de la bici al iniciar o cerrar un viaje, o al terminar un mantenimiento, no está en la base: lo tendría que hacer la aplicación.
3. Los indicadores se calculan con vistas y no se guardan, como los atributos derivados de la clase S02.

Pruebas. Primero las hice en PostgreSQL 17 local y después corrí el script, los datos de ejemplo y la prueba B1 en Supabase, con el mismo resultado:

- **Resultados con los datos de ejemplo:**
  - la vista de disponibilidad marca E03 como LLENA y E01 y E02 como VACIA;
  - la de repuestos muestra R-CAMARA (stock 3, igual al mínimo);
  - al usar 2 pastillas de freno, el stock bajó de 10 a 8;
  - los indicadores dan 2 viajes terminados, 32,5 minutos de promedio, 11,1 % de las bicis en mantenimiento y 1 falla abierta.
- **La base rechazó:**
  - usar más repuestos que el stock;
  - un traslado de E01 a E01;
  - un umbral mayor que la capacidad;
  - un reporte de un viaje que no existe;
  - un tipo de mantenimiento “URGENTE”.

Capturas de Supabase.



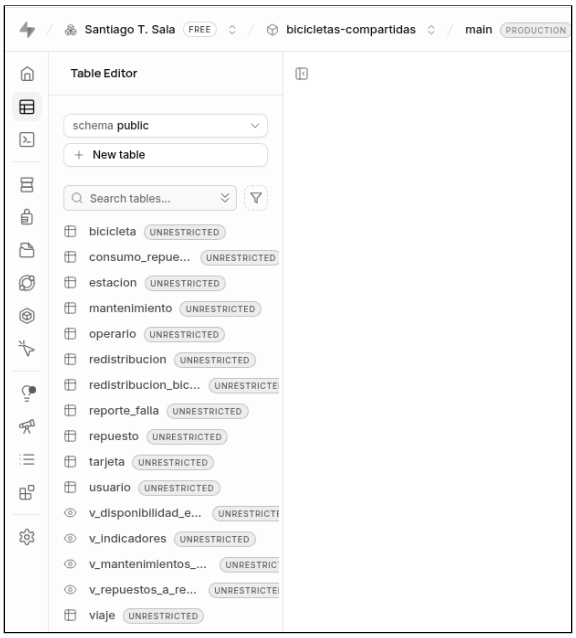
Captura 1. Script etapa2_crear_base.sql ejecutado sin errores.



Captura 2. Datos de ejemplo cargados; la vista v_indicadores da 2 viajes, 32,5 min, 11,1 % y 3 estaciones con alerta.



Captura 3. Prueba B1: la base no deja usar 5 cámaras si hay 3 (el stock quedaría en −2).



Captura 4. Las 12 tablas y las 4 vistas en el Table Editor (todavía sin RLS: “UNRESTRICTED”).

5. Aplicación web con IA

Pedido a la IA. Usé como prompt el documento descriptivo completo (punto 2) y le agregué al final:

> Con este documento generá una aplicación web en español conectada a mi base de Supabase (el esquema es el de sql/etapa2_crear_base.sql, con sus vistas), pensada para publicarse en Vercel. Pantallas mínimas: panel con indicadores, estaciones con disponibilidad y alertas, bicicletas y estados, registrar retiro y devolución de viaje, redistribución de flota, mantenimiento y reportes de falla, y repuestos. Estilo sobrio en blanco y negro, que funcione bien en el celular. Las reglas que ya controla la base (capacidad, estados, stock) no las repitas: mostrá el error de la base en castellano.

Respuesta. La generé con el mismo asistente de IA que usé para el SQL (Grok Bot). El código completo está en la carpeta app del repositorio.

Correcciones que le pedí:

1. *Cambio de tecnología.* El documento decía Next.js, pero le pedí una aplicación estática (HTML, CSS y JavaScript con supabase-js desde un CDN), porque no necesita instalación ni compilación, la puedo explicar archivo por archivo y Vercel la publica tal cual.
2. *Los botones de la pantalla Viajes no hacían nada.* El código conectaba los formularios antes de que la pantalla estuviera dibujada. Le pedí corregirlo: ahora cada pantalla conecta sus botones recién cuando ya se muestra.
3. *Estación de retiro.* Por defecto aparecía una estación sin bicis disponibles. Le pedí que proponga la que tiene más.
4. *Números.* Mostraba 32.5; le pedí el formato argentino (32,5).
5. *Orden de los pasos al devolver.* Primero se mueve la bici a la estación de destino (si está llena, la base lo rechaza y no se cierra el viaje) y, si no se puede cerrar el viaje, la bici vuelve a como estaba. Lo mismo al registrar la llegada de un traslado.
6. *Seguridad.* La clave anon es pública y las tablas estaban sin RLS. Le pedí un script aparte, sql/etapa2_rls.sql (Anexo C), con permisos básicos: se puede leer todo, dar de alta y modificar solo donde la app registra movimientos, y no se puede borrar nada.

Cómo queda cada función en la app:

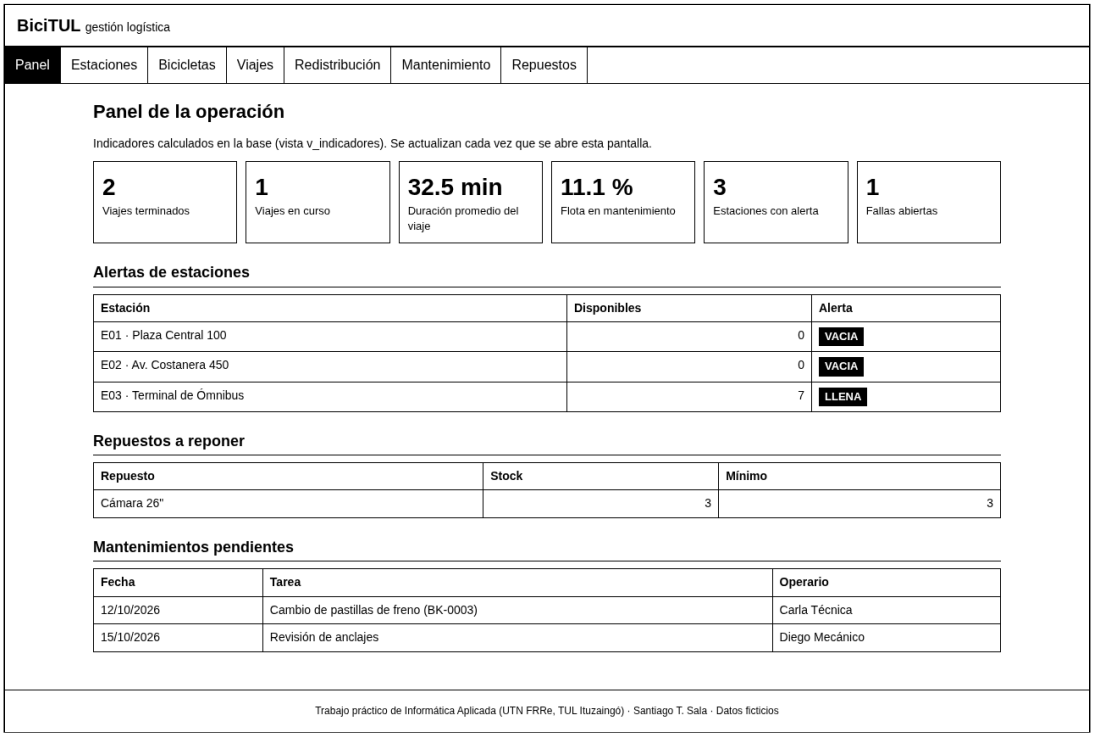
Pantalla	Función	Qué hace
Panel	F5	Indicadores, alertas de estaciones, repuestos a reponer y tareas pendientes
Estaciones	F1	Disponibles, asignadas, capacidad, umbrales y alerta VACIA / OK / LLENA
Bicicletas	—	Lista con filtro por estado y alta de bicis
Viajes	—	Retiro (la bici pasa a EN_USO) y devolución; si informo una falla se crea el reporte y un mantenimiento CORRECTIVO
Redistribución	F2	Orden de traslado sugerida de una estación LLENA a una VACIA; salida (EN_TRASLADO) y llegada (cambia de estación y vuelve a DISPONIBLE)
Mantenimiento	F3	

Pantalla	Función	Qué hace
		Pendientes, programar tareas y marcar como realizadas (la bici vuelve a DISPONIBLE y el reporte queda RESUELTO)
Repuestos	F4	Stock con alerta REPONER, consumo (la base descuenta el stock), ingreso de compras y alta

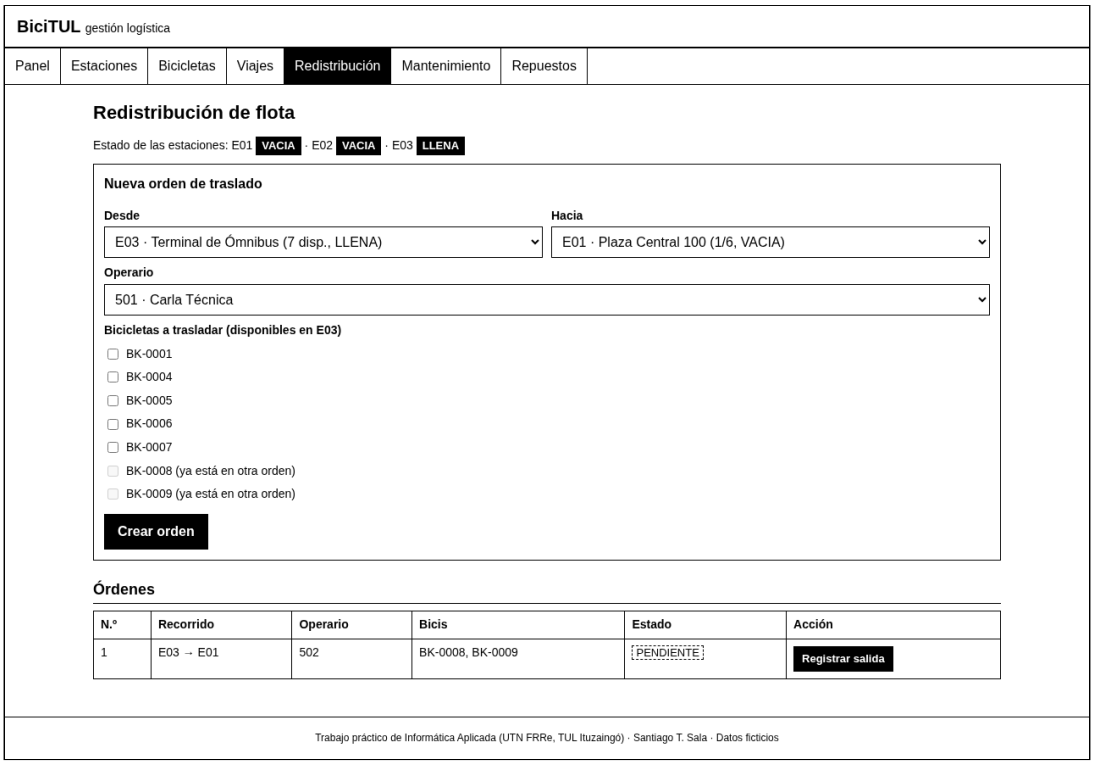
Pruebas.

- *Contra mi base real de Supabase:* retiré BK-0004 en E03, la devolví en E01 y completé la orden de traslado N.º 1 (BK-0008 y BK-0009 de E03 a E01). E01 y E03 pasaron de VACIA y LLENA a OK. Esto cambió los datos de ejemplo: ahora hay 3 viajes terminados y el promedio bajó a 21,7 minutos, porque el viaje de prueba duró segundos.
- *En una copia local de la base, con el script de RLS ya aplicado:* devolución con falla (se creó el reporte y el correctivo), mantenimiento realizado, consumo de repuestos y descuento de stock, ingreso de stock, alta de bici y de orden de traslado. La app mostró bien los rechazos: más repuestos que el stock, traslado a la misma estación y estación llena. Con RLS no se pudo borrar un viaje ni cambiar una estación desde la clave anon.

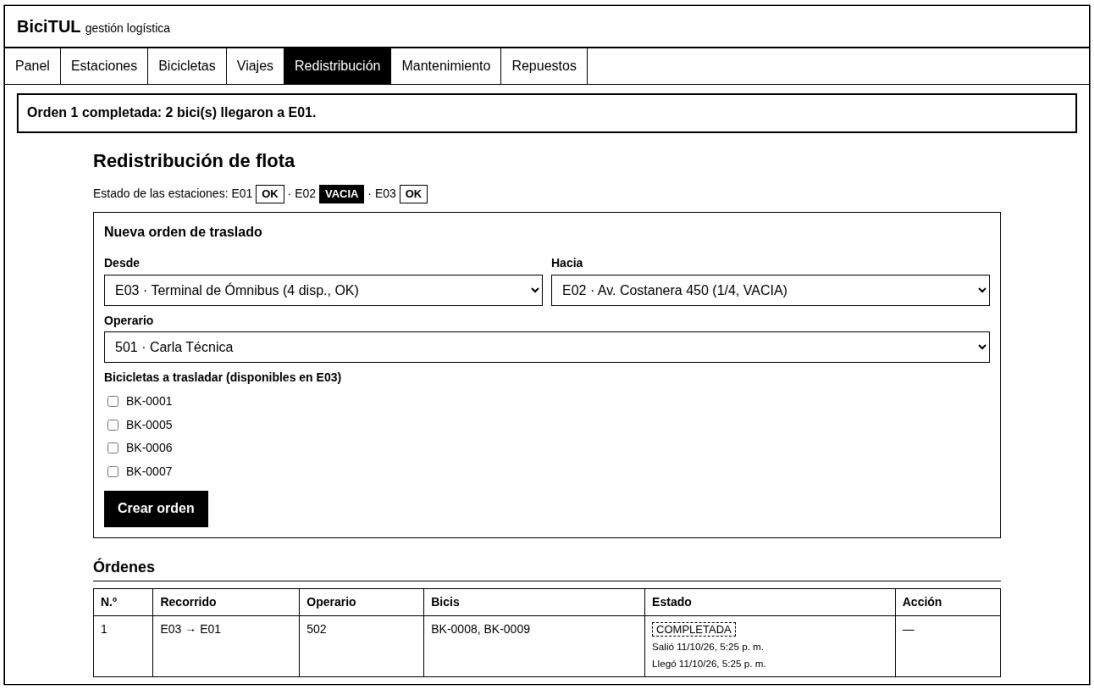
Capturas de la aplicación.



Captura 5. Panel con los indicadores y alertas de los datos de ejemplo (escritorio).



Captura 6. Redistribución: propone de E03 (LLENA) a E01 (VACIA); la orden N.º 1 está pendiente.



Captura 7. Después de registrar la salida y la llegada de la orden N.º 1: E01 y E03 quedan OK.



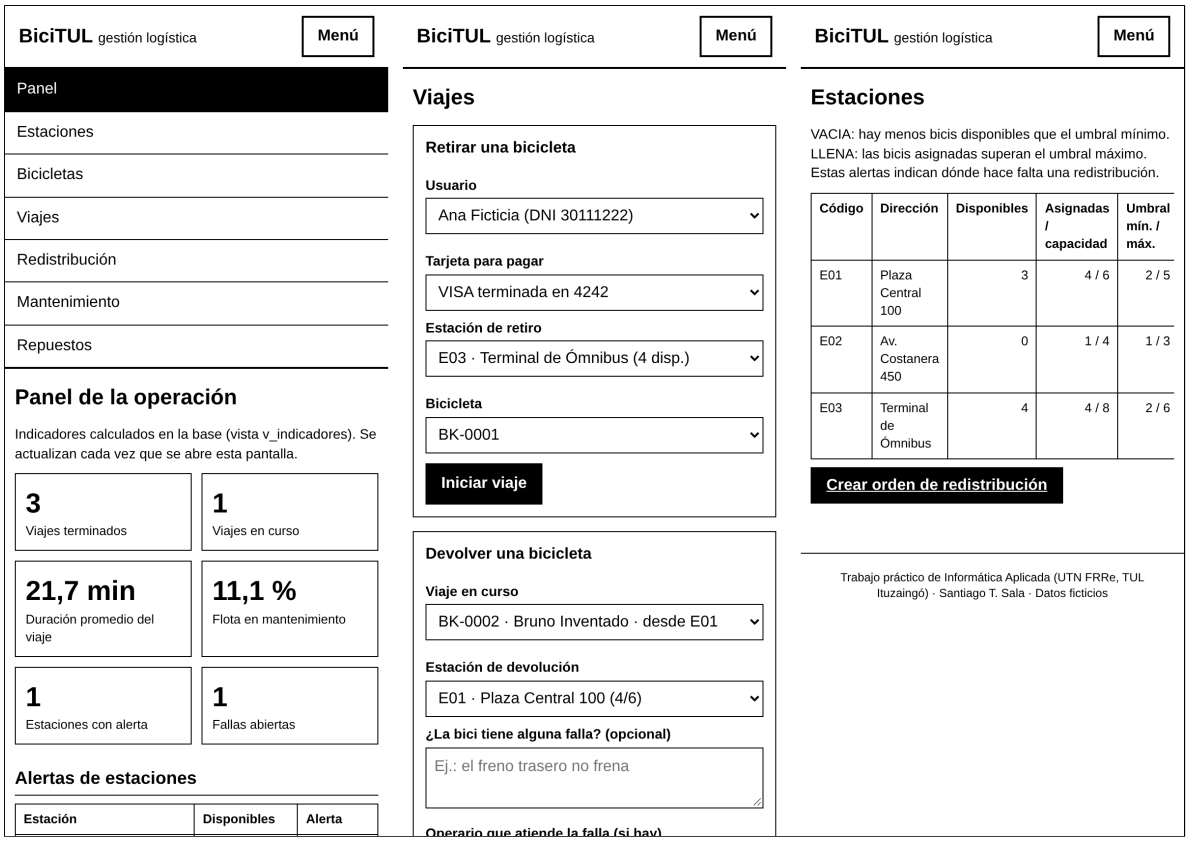
Captura 8. Viajes: retiro, devolución (con reporte de falla opcional) y últimos viajes.



Captura 9. Mantenimiento: tareas pendientes, programar, reportes de falla y realizados.



Captura 10. Repuestos: stock con alerta REPONER y registro de consumos.



Captura 11. En el celular: menú, pantalla Viajes y Estaciones (esta última, desde la dirección publicada en Vercel).

Publicación en Vercel. Creé el proyecto bicicletas-compartidas y publiqué los archivos estáticos. Dirección: **https://bicicletas-compartidas.vercel.app**. Por ahora la subí directamente (no está conectada a GitHub); cuando el profesor la importe desde el repositorio, tiene que elegir la carpeta app como Root Directory (los pasos están en app/README.md).

Lo que falta o mejoraría.

- Correr sql/etapa2_rls.sql en Supabase (Anexo C) y comprobar que la app siga funcionando.
- No hay inicio de sesión ni roles: cualquiera que tenga el link puede usarla. Para que cada rol (usuario, operario, supervisor) haga solo lo suyo, como piden las reglas del negocio, hace falta Supabase Auth y políticas por rol.
- Algunas operaciones tienen varios pasos (por ejemplo, mover la bici y cerrar el viaje). Si se corta internet en el medio podrían quedar a medias; lo correcto sería pasarlas a funciones de la base, que se ejecutan en una sola transacción.

6. Repositorio

Enlace: https://github.com/jumptowin/informatica-aplicada-tul. Contiene:

- README.md y .gitignore (que excluye .env);
- etapa1/: la entrega de la primera etapa;
- etapa2/: esta entrega con la documentación (documento descriptivo dentro de este PDF, MER, MR y SQL, incluido el de RLS) y las capturas;

- app/: el código de la aplicación, con su README.md (qué hace, qué funciones cubre y qué variables de entorno necesita), .env.example sin valores y config.js con la clave anon, que es pública.

Acceso al docente: falta el usuario del profesor (en la consigna figura “[completar]”).

Anexo A — sql/etapa2_crear_base.sql

```
-- ======================================================================
-- Bicicletas compartidas - ETAPA 2 - Script COMPLETO (recrea la base desde cero)
-- PostgreSQL / Supabase SQL Editor. Solo datos ficticios.
-- Decisión: se recrea desde cero porque la base de la Etapa 1 solo tiene datos de prueba.
-- (Si hubiera datos reales habría que usar ALTER TABLE para no perderlos.)
-- ======================================================================
DROP VIEW  IF EXISTS v_disponibilidad_estacion, v_repuestos_a_reponer,
v_mantenimientos_pendientes, v_indicadores CASCADE;
DROP TABLE IF EXISTS redistribucion_bicicleta, redistribucion, consumo_repuesto, repuesto,
                     reporte_falla, viaje, mantenimiento, tarjeta, bicicleta, operario, usuario,
estacion CASCADE;
DROP FUNCTION IF EXISTS controlar_capacidad_estacion() CASCADE;
DROP FUNCTION IF EXISTS descontar_stock_repuesto() CASCADE;

CREATE TABLE estacion (
  codigo         VARCHAR(10)  PRIMARY KEY,
  direccion      VARCHAR(150) NOT NULL,
  capacidad_max  INTEGER      NOT NULL CHECK (capacidad_max > 0),
  latitud        NUMERIC(9,6),                                    -- NUEVO
  longitud       NUMERIC(9,6),                                    -- NUEVO
  umbral_minimo  INTEGER NOT NULL DEFAULT 1,                      -- NUEVO: menos bicis que esto
= alerta "vacía"
  umbral_maximo  INTEGER,                                         -- NUEVO: más bicis que esto =
alerta "llena"
  CHECK (umbral_minimo >= 0 AND (umbral_maximo IS NULL OR (umbral_maximo <= capacidad_max AND
umbral_maximo >= umbral_minimo)))
);

CREATE TABLE bicicleta (
  numero_serie     VARCHAR(30) PRIMARY KEY,
  estado           VARCHAR(20) NOT NULL DEFAULT 'DISPONIBLE'
                   CHECK (estado IN ('DISPONIBLE','EN_USO','EN_MANTENIMIENTO','EN_TRASLADO')),
-- MODIFICADO
  codigo_estacion  VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  fecha_alta       DATE NOT NULL DEFAULT CURRENT_DATE                                    --
NUEVO
);

CREATE TABLE usuario (
  dni     VARCHAR(8)   PRIMARY KEY CHECK (dni ~ '^[0-9]{7,8}$'),
  nombre  VARCHAR(120) NOT NULL
);

CREATE TABLE tarjeta (
  id_tarjeta         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dni_usuario        VARCHAR(8) NOT NULL REFERENCES usuario(dni) ON DELETE CASCADE,
  marca              VARCHAR(30) NOT NULL,
  ultimos_4_digitos  CHAR(4) NOT NULL CHECK (ultimos_4_digitos ~ '^[0-9]{4}$'),
  vencimiento        CHAR(5) NOT NULL CHECK (vencimiento ~ '^(0[1-9]|1[0-2])/[0-9]{2}$'),
  UNIQUE (id_tarjeta, dni_usuario)
);

CREATE TABLE operario (
  legajo  INTEGER PRIMARY KEY CHECK (legajo > 0),
  nombre  VARCHAR(120) NOT NULL
);

CREATE TABLE viaje (
  dni_usuario              VARCHAR(8)  NOT NULL REFERENCES usuario(dni),
  numero_serie             VARCHAR(30) NOT NULL REFERENCES bicicleta(numero_serie),
  fecha_hora_inicio        TIMESTAMPTZ NOT NULL,
  fecha_hora_fin           TIMESTAMPTZ,
  codigo_estacion_origen   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  codigo_estacion_destino  VARCHAR(10) REFERENCES estacion(codigo),
  id_tarjeta               INTEGER NOT NULL,
```

```
  PRIMARY KEY (dni_usuario, numero_serie, fecha_hora_inicio),
  FOREIGN KEY (id_tarjeta, dni_usuario) REFERENCES tarjeta(id_tarjeta, dni_usuario),
  CHECK (fecha_hora_fin IS NULL OR fecha_hora_fin > fecha_hora_inicio),
  CHECK ((fecha_hora_fin IS NULL) = (codigo_estacion_destino IS NULL))
);
CREATE UNIQUE INDEX un_viaje_abierto_por_bici    ON viaje (numero_serie) WHERE fecha_hora_fin IS
NULL;
CREATE UNIQUE INDEX un_viaje_abierto_por_usuario ON viaje (dni_usuario)  WHERE fecha_hora_fin IS
NULL;

-- NUEVA: reporte de falla hecho por el usuario al terminar un viaje (N:1 con VIAJE)
CREATE TABLE reporte_falla (
  id_reporte         INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  dni_usuario        VARCHAR(8)  NOT NULL,
  numero_serie       VARCHAR(30) NOT NULL,
  fecha_hora_inicio  TIMESTAMPTZ NOT NULL,
  fecha_hora         TIMESTAMPTZ NOT NULL DEFAULT now(),
  descripcion        TEXT NOT NULL,
  estado             VARCHAR(10) NOT NULL DEFAULT 'ABIERTO' CHECK (estado IN
('ABIERTO','RESUELTO')),
  FOREIGN KEY (dni_usuario, numero_serie, fecha_hora_inicio)
    REFERENCES viaje(dni_usuario, numero_serie, fecha_hora_inicio)
);

-- MODIFICADA: tipo, estado y bicicleta (opcional)
CREATE TABLE mantenimiento (
  id_mantenimiento  INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  fecha             DATE NOT NULL,
  descripcion       TEXT NOT NULL,
  codigo_estacion   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  legajo_operario   INTEGER     NOT NULL REFERENCES operario(legajo),
  tipo              VARCHAR(12) NOT NULL DEFAULT 'PREVENTIVO' CHECK (tipo IN
('PREVENTIVO','CORRECTIVO')),   -- NUEVO
  estado            VARCHAR(12) NOT NULL DEFAULT 'PROGRAMADO' CHECK (estado IN
('PROGRAMADO','REALIZADO')),  -- NUEVO
  numero_serie      VARCHAR(30) REFERENCES
bicicleta(numero_serie)                                          -- NUEVO (NULL = tarea de la
estación)
);

-- NUEVAS: repuestos y su consumo (N:M MANTENIMIENTO-REPUESTO con atributo cantidad)
CREATE TABLE repuesto (
  codigo_repuesto  VARCHAR(20) PRIMARY KEY,
  descripcion      VARCHAR(120) NOT NULL,
  stock_actual     INTEGER NOT NULL DEFAULT 0 CHECK (stock_actual >= 0),
  stock_minimo     INTEGER NOT NULL DEFAULT 0 CHECK (stock_minimo >= 0)
);
CREATE TABLE consumo_repuesto (
  id_mantenimiento  INTEGER     NOT NULL REFERENCES mantenimiento(id_mantenimiento) ON DELETE
CASCADE,
  codigo_repuesto   VARCHAR(20) NOT NULL REFERENCES repuesto(codigo_repuesto),
  cantidad          INTEGER     NOT NULL CHECK (cantidad > 0),
  PRIMARY KEY (id_mantenimiento, codigo_repuesto)
);

-- NUEVAS: redistribución de flota (orden de traslado) y bicis trasladadas (N:M)
CREATE TABLE redistribucion (
  id_redistribucion       INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  codigo_estacion_desde   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  codigo_estacion_hacia   VARCHAR(10) NOT NULL REFERENCES estacion(codigo),
  legajo_operario         INTEGER     NOT NULL REFERENCES operario(legajo),
  fecha_hora_salida       TIMESTAMPTZ,
  fecha_hora_llegada      TIMESTAMPTZ,
  estado                  VARCHAR(12) NOT NULL DEFAULT 'PENDIENTE' CHECK (estado IN
('PENDIENTE','EN_CURSO','COMPLETADA')),
  CHECK (codigo_estacion_desde <> codigo_estacion_hacia),
  CHECK (fecha_hora_llegada IS NULL OR fecha_hora_llegada > fecha_hora_salida)
);
CREATE TABLE redistribucion_bicicleta (
  id_redistribucion  INTEGER     NOT NULL REFERENCES redistribucion(id_redistribucion) ON DELETE
CASCADE,
  numero_serie       VARCHAR(30) NOT NULL REFERENCES bicicleta(numero_serie),
  PRIMARY KEY (id_redistribucion, numero_serie)
);

-- Regla de capacidad (igual que Etapa 1)
```

```
CREATE FUNCTION controlar_capacidad_estacion() RETURNS trigger AS $$
BEGIN
  IF (SELECT COUNT(*) FROM bicicleta WHERE codigo_estacion = NEW.codigo_estacion)
     > (SELECT capacidad_max FROM estacion WHERE codigo = NEW.codigo_estacion) THEN
    RAISE EXCEPTION 'La estacion % supera su capacidad maxima', NEW.codigo_estacion;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_capacidad_estacion AFTER INSERT OR UPDATE OF codigo_estacion ON bicicleta
FOR EACH ROW EXECUTE FUNCTION controlar_capacidad_estacion();

-- Regla: usar un repuesto descuenta stock (y falla si no alcanza, por el CHECK stock_actual >=
0)
CREATE FUNCTION descontar_stock_repuesto() RETURNS trigger AS $$
BEGIN
  UPDATE repuesto SET stock_actual = stock_actual - NEW.cantidad WHERE codigo_repuesto =
NEW.codigo_repuesto;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;
CREATE TRIGGER trg_descontar_stock AFTER INSERT ON consumo_repuesto
FOR EACH ROW EXECUTE FUNCTION descontar_stock_repuesto();

-- ===== Vistas para las funciones logísticas =====
-- F1 Disponibilidad y alertas por estación
CREATE VIEW v_disponibilidad_estacion AS
SELECT e.codigo, e.direccion, e.capacidad_max, e.umbral_minimo, e.umbral_maximo,
       COUNT(b.numero_serie) FILTER (WHERE b.estado = 'DISPONIBLE') AS bicis_disponibles,
       COUNT(b.numero_serie)                                          AS bicis_asignadas,
       CASE WHEN COUNT(b.numero_serie) FILTER (WHERE b.estado='DISPONIBLE') < e.umbral_minimo
THEN 'VACIA'
            WHEN e.umbral_maximo IS NOT NULL AND COUNT(b.numero_serie) > e.umbral_maximo THEN
'LLENA'
            ELSE 'OK' END AS alerta
FROM estacion e LEFT JOIN bicicleta b ON b.codigo_estacion = e.codigo
GROUP BY e.codigo;
-- F3 Mantenimientos pendientes
CREATE VIEW v_mantenimientos_pendientes AS
SELECT m.*, o.nombre AS operario FROM mantenimiento m JOIN operario o ON o.legajo =
m.legajo_operario
WHERE m.estado = 'PROGRAMADO' ORDER BY m.fecha;
-- F4 Repuestos a reponer
CREATE VIEW v_repuestos_a_reponer AS
SELECT * FROM repuesto WHERE stock_actual <= stock_minimo;
-- F5 Indicadores
CREATE VIEW v_indicadores AS
SELECT
  (SELECT COUNT(*) FROM viaje WHERE fecha_hora_fin IS NOT
NULL)                                  AS viajes_terminados,
  (SELECT ROUND(AVG(EXTRACT(EPOCH FROM (fecha_hora_fin - fecha_hora_inicio))/60)::numeric,1)
     FROM viaje WHERE fecha_hora_fin IS NOT
NULL)                                                AS duracion_promedio_min,
  (SELECT ROUND(100.0*COUNT(*) FILTER (WHERE estado='EN_MANTENIMIENTO')/NULLIF(COUNT(*),0),1)
FROM bicicleta) AS pct_flota_en_mantenimiento,
  (SELECT COUNT(*) FROM v_disponibilidad_estacion WHERE alerta <>
'OK')                             AS estaciones_con_alerta,
  (SELECT COUNT(*) FROM reporte_falla WHERE
estado='ABIERTO')                                     AS fallas_abiertas;
```

Anexo B — sql/etapa2_datos_y_pruebas.sql

```
-- ETAPA 2 - Datos ficticios (PARTE A, correr junta) y pruebas (PARTE B, una por vez: todas
deben dar error)
-- ---------- PARTE A ----------
INSERT INTO estacion VALUES
 ('E01','Plaza Central 100',  6, -27.4690,-58.8300, 2, 5),
 ('E02','Av. Costanera 450',  4, -27.4600,-58.8400, 1, 3),
 ('E03','Terminal de Ómnibus',8, -27.4800,-58.8200, 2, 6);
INSERT INTO bicicleta (numero_serie, estado, codigo_estacion, fecha_alta) VALUES
 ('BK-0001','DISPONIBLE','E03','2025-03-01'),
 ('BK-0002','EN_USO','E01','2025-03-01'),
 ('BK-0003','EN_MANTENIMIENTO','E02','2025-05-10'),
 ('BK-0004','DISPONIBLE','E03','2025-05-10'),
 ('BK-0005','DISPONIBLE','E03','2026-01-15'),
 ('BK-0006','DISPONIBLE','E03','2026-01-15'),
 ('BK-0007','DISPONIBLE','E03','2026-01-15'),
```

```
 ('BK-0008','DISPONIBLE','E03','2026-01-15'),
 ('BK-0009','DISPONIBLE','E03','2026-01-15');
INSERT INTO usuario VALUES ('30111222','Ana Ficticia'), ('28999888','Bruno Inventado');
INSERT INTO tarjeta (dni_usuario, marca, ultimos_4_digitos, vencimiento) VALUES
 ('30111222','VISA','4242','08/28'), ('30111222','MASTERCARD','5555','01/27'),
('28999888','VISA','1111','12/29');
INSERT INTO operario VALUES (501,'Carla Técnica'), (502,'Diego Mecánico');
INSERT INTO viaje VALUES
 ('30111222','BK-0001','2026-10-10 09:00-03','2026-10-10 09:25-03','E01','E03',1),
 ('30111222','BK-0003','2026-10-10 17:00-03','2026-10-10 17:40-03','E03','E02',2),
 ('28999888','BK-0002','2026-10-11 18:00-03',NULL,'E01',NULL,3);
INSERT INTO reporte_falla (dni_usuario, numero_serie, fecha_hora_inicio, descripcion)
VALUES ('30111222','BK-0003','2026-10-10 17:00-03','Freno trasero no frena');
INSERT INTO repuesto VALUES ('R-FRENO','Pastilla de freno',10,4), ('R-CAMARA','Cámara 26"',3,3);
INSERT INTO mantenimiento (fecha, descripcion, codigo_estacion, legajo_operario, tipo, estado,
numero_serie) VALUES
 ('2026-10-12','Cambio de pastillas de freno','E02',501,'CORRECTIVO','PROGRAMADO','BK-0003'),
 ('2026-10-15','Revisión de anclajes','E02',502,'PREVENTIVO','PROGRAMADO',NULL);
INSERT INTO consumo_repuesto VALUES (1,'R-FRENO',2);
INSERT INTO redistribucion (codigo_estacion_desde, codigo_estacion_hacia, legajo_operario,
estado)
VALUES ('E03','E01',502,'PENDIENTE');
INSERT INTO redistribucion_bicicleta VALUES (1,'BK-0008'), (1,'BK-0009');

SELECT * FROM v_disponibilidad_estacion ORDER BY codigo;
SELECT * FROM v_repuestos_a_reponer;
SELECT * FROM v_mantenimientos_pendientes;
SELECT * FROM v_indicadores;

-- ---------- PARTE B: deben dar ERROR ----------
-- B1 usar más repuestos que el stock (R-CAMARA tiene 3)
INSERT INTO consumo_repuesto VALUES (2,'R-CAMARA',5);
-- B2 redistribución desde y hacia la misma estación
INSERT INTO redistribucion (codigo_estacion_desde, codigo_estacion_hacia, legajo_operario)
VALUES ('E01','E01',501);
-- B3 umbral máximo mayor que la capacidad
INSERT INTO estacion VALUES ('E09','Prueba',4,NULL,NULL,1,9);
-- B4 reporte de falla de un viaje que no existe
INSERT INTO reporte_falla (dni_usuario, numero_serie, fecha_hora_inicio, descripcion) VALUES
('30111222','BK-0004','2026-01-01 10:00-03','x');
-- B5 tipo de mantenimiento no permitido
INSERT INTO mantenimiento (fecha, descripcion, codigo_estacion, legajo_operario, tipo) VALUES
('2026-10-20','x','E01',501,'URGENTE');
```

Anexo C — sql/etapa2_rls.sql

```
-- ======================================================================
-- Bicicletas compartidas - ETAPA 2 - Seguridad por filas (RLS) básica
-- Correr en Supabase > SQL Editor DESPUÉS de etapa2_crear_base.sql
-- (y de cargar los datos). Se puede volver a correr: primero borra las políticas.
--
-- Por qué: la aplicación usa la clave pública "anon", que cualquiera puede ver
-- en el navegador. Sin RLS, con esa clave se puede leer, modificar y BORRAR
-- todo. Con este script:
--   * RLS queda activado en las 12 tablas;
--   * el rol anon solo puede LEER todas las tablas que muestra la app;
--   * solo puede INSERTAR/MODIFICAR en las tablas donde la app registra
--     movimientos (bicicleta, viaje, reporte_falla, mantenimiento, repuesto,
--     consumo_repuesto, redistribucion, redistribucion_bicicleta);
--   * estacion, usuario, tarjeta y operario quedan de solo lectura;
--   * NADIE puede borrar desde la app (no hay política de DELETE).
-- Límite: sigue sin haber usuarios ni roles (usuario / operario / supervisor).
-- Para eso haría falta Supabase Auth y políticas por rol (mejora pendiente).
-- ======================================================================

-- 1) Activar RLS en todas las tablas
ALTER TABLE estacion                ENABLE ROW LEVEL SECURITY;
ALTER TABLE bicicleta               ENABLE ROW LEVEL SECURITY;
ALTER TABLE usuario                 ENABLE ROW LEVEL SECURITY;
ALTER TABLE tarjeta                 ENABLE ROW LEVEL SECURITY;
ALTER TABLE operario                ENABLE ROW LEVEL SECURITY;
ALTER TABLE viaje                   ENABLE ROW LEVEL SECURITY;
ALTER TABLE reporte_falla           ENABLE ROW LEVEL SECURITY;
ALTER TABLE mantenimiento           ENABLE ROW LEVEL SECURITY;
ALTER TABLE repuesto                ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE consumo_repuesto         ENABLE ROW LEVEL SECURITY;
ALTER TABLE redistribucion           ENABLE ROW LEVEL SECURITY;
ALTER TABLE redistribucion_bicicleta ENABLE ROW LEVEL SECURITY;

-- 2) Borrar políticas anteriores (para poder correr el script dos veces)
DO $$
DECLARE p record;
BEGIN
  FOR p IN SELECT policyname, tablename FROM pg_policies
           WHERE schemaname = 'public' AND policyname LIKE 'app_%' LOOP
    EXECUTE format('DROP POLICY %I ON public.%I', p.policyname, p.tablename);
  END LOOP;
END $$;

-- 3) LECTURA: la app muestra todas estas tablas
CREATE POLICY app_leer ON estacion                FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON bicicleta               FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON usuario                 FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON tarjeta                 FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON operario                FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON viaje                   FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON reporte_falla           FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON mantenimiento           FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON repuesto                FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON consumo_repuesto        FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON redistribucion          FOR SELECT TO anon, authenticated USING
(true);
CREATE POLICY app_leer ON redistribucion_bicicleta FOR SELECT TO anon, authenticated USING
(true);

-- 4) ALTA (INSERT): solo donde la app registra movimientos
CREATE POLICY app_alta ON bicicleta               FOR INSERT TO anon, authenticated WITH CHECK
(true);  -- alta de bici
CREATE POLICY app_alta ON viaje                   FOR INSERT TO anon, authenticated WITH CHECK
(fecha_hora_fin IS NULL);  -- un viaje nace abierto
CREATE POLICY app_alta ON reporte_falla           FOR INSERT TO anon, authenticated WITH CHECK
(estado = 'ABIERTO');
CREATE POLICY app_alta ON mantenimiento           FOR INSERT TO anon, authenticated WITH CHECK
(estado = 'PROGRAMADO');
CREATE POLICY app_alta ON repuesto                FOR INSERT TO anon, authenticated WITH CHECK
(true);
CREATE POLICY app_alta ON consumo_repuesto        FOR INSERT TO anon, authenticated WITH CHECK
(true);
CREATE POLICY app_alta ON redistribucion          FOR INSERT TO anon, authenticated WITH CHECK
(estado = 'PENDIENTE');
CREATE POLICY app_alta ON redistribucion_bicicleta FOR INSERT TO anon, authenticated WITH CHECK
(true);

-- 5) CAMBIOS (UPDATE): estados, cierre de viajes, stock y órdenes
CREATE POLICY app_cambio ON bicicleta      FOR UPDATE TO anon, authenticated USING (true) WITH
CHECK (true);
CREATE POLICY app_cambio ON viaje          FOR UPDATE TO anon, authenticated USING
(fecha_hora_fin IS NULL) WITH CHECK (true);  -- solo se cierran viajes abiertos
CREATE POLICY app_cambio ON reporte_falla  FOR UPDATE TO anon, authenticated USING (true) WITH
CHECK (true);
CREATE POLICY app_cambio ON mantenimiento  FOR UPDATE TO anon, authenticated USING (true) WITH
CHECK (true);
CREATE POLICY app_cambio ON repuesto       FOR UPDATE TO anon, authenticated USING (true) WITH
CHECK (true);  -- también lo usa el trigger de stock
CREATE POLICY app_cambio ON redistribucion FOR UPDATE TO anon, authenticated USING (estado <>
'COMPLETADA') WITH CHECK (true);

-- 6) Sin borrados ni vaciados desde la API
REVOKE TRUNCATE ON ALL TABLES IN SCHEMA public FROM anon, authenticated;

-- 7) Las vistas respetan RLS de quien consulta (sin esto, una vista se
```

```
--    ejecuta con los permisos de su dueño y "saltea" la seguridad).
ALTER VIEW v_disponibilidad_estacion   SET (security_invoker = true);
ALTER VIEW v_mantenimientos_pendientes SET (security_invoker = true);
ALTER VIEW v_repuestos_a_reponer       SET (security_invoker = true);
ALTER VIEW v_indicadores               SET (security_invoker = true);

-- 8) Control: debería listar 12 tablas con rowsecurity = true
SELECT tablename, rowsecurity FROM pg_tables WHERE schemaname = 'public' ORDER BY tablename;
```